ENTERPRISE SECURITY ASSESSMENT LAB

GRAPHQL ENDPOINT TESTING, INTROSPECTION, AND USER DATA QUERY REVIEW

DVGA GraphQL lab validation, schema discovery, and resolver-level authorization analysis

Field	Details
Module	Sub-Project 2 - API Security Testing Lab
Document Type	Individual API Security Methodology PDF
Phase / Test Case	Phase 9 - GraphQL Testing
Prepared For	GitHub Portfolio Documentation
Prepared By	Jagriti Banerjee
Repository	Enterprise-Security-Assessment-Lab
GitHub Filename	09-graphql-testing-methodology-report.pdf

This document is a professional, evidence-driven explanation of one API security methodology section. It is designed for portfolio review and contains only authorized lab-based testing context. Sensitive token values, account data, cookies, and private identifiers are redacted or intentionally represented with placeholders.

1. Section Overview

Area	Details
Phase	Phase 9 - GraphQL Testing
Objective	Validate GraphQL endpoint availability, test introspection behaviour, and review how user-data queries behave in the lab environment.
Primary Result	Evidence captured and methodology documented
GitHub Folder	02-api-security/reports/methodology/

Why this section matters

GraphQL testing requires a different mindset from REST testing. A single endpoint can expose many types, queries, mutations, and nested relationships. Introspection can make mapping easier, but resolver-level authorization determines whether sensitive data is actually protected.

2. Evidence Embedded in This PDF

Evidence 1: 11-dvga-graphql-lab-running.png

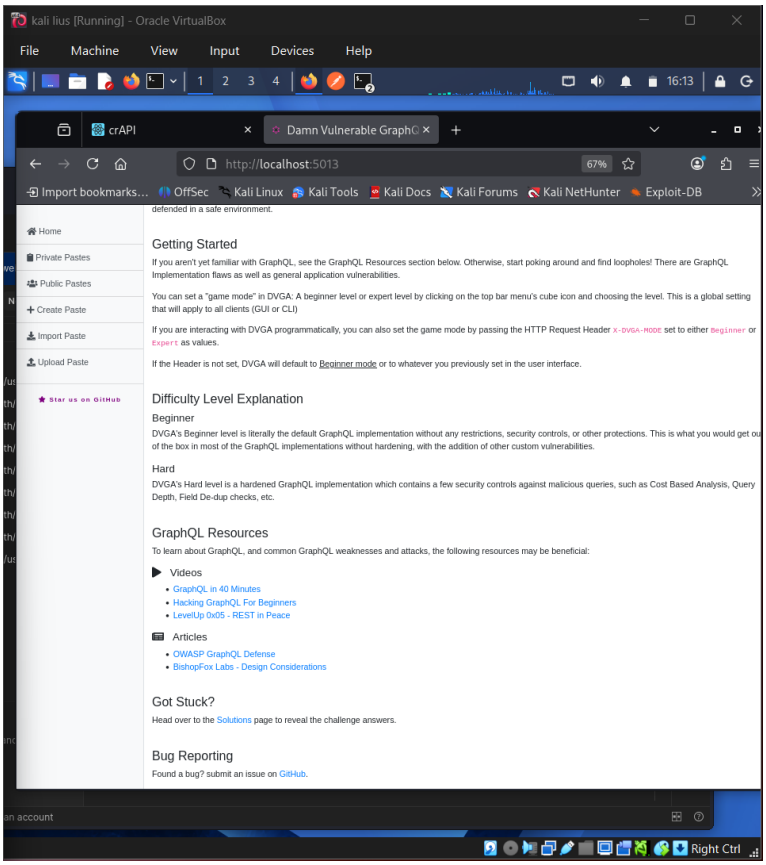

Figure 1 - Evidence that the DVGA GraphQL lab was running.

Evidence that the DVGA GraphQL lab was running.

Evidence 2: 11-graphql-endpoint-alive.png

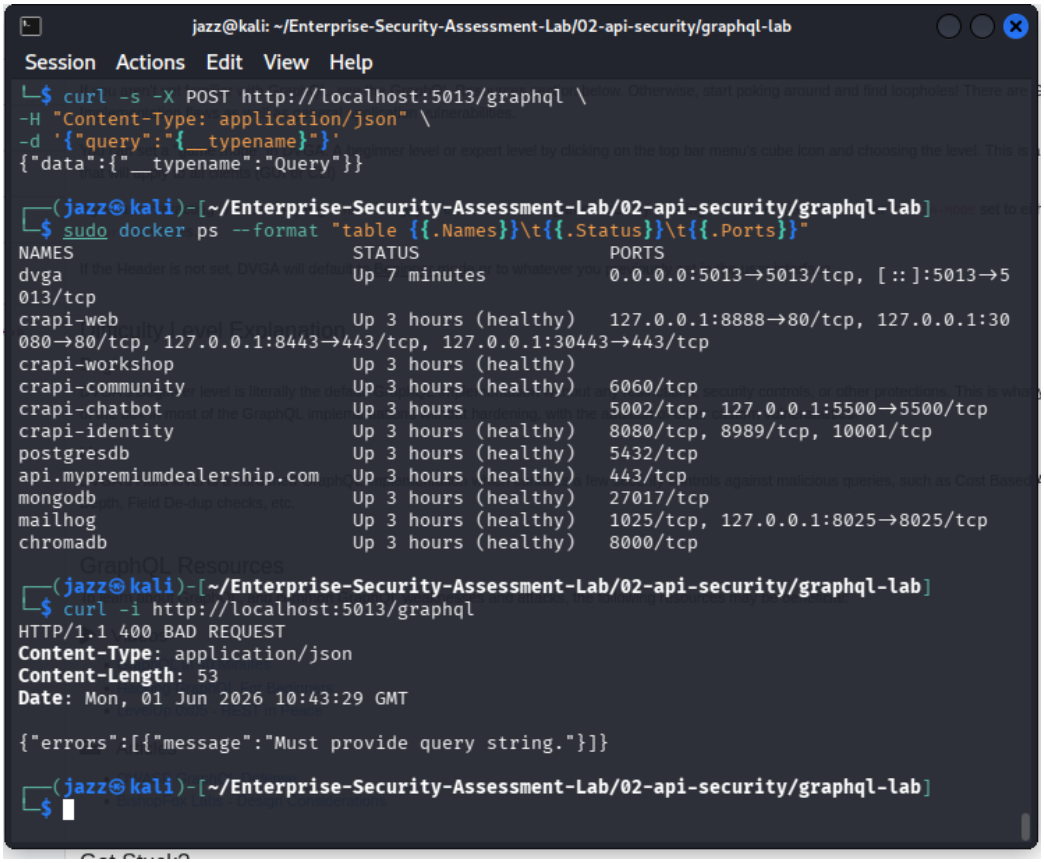

Figure 2 - Evidence that the GraphQL endpoint was reachable.

Evidence that the GraphQL endpoint was reachable.

Evidence 3: 12-graphql-introspection-schema-discovery.png

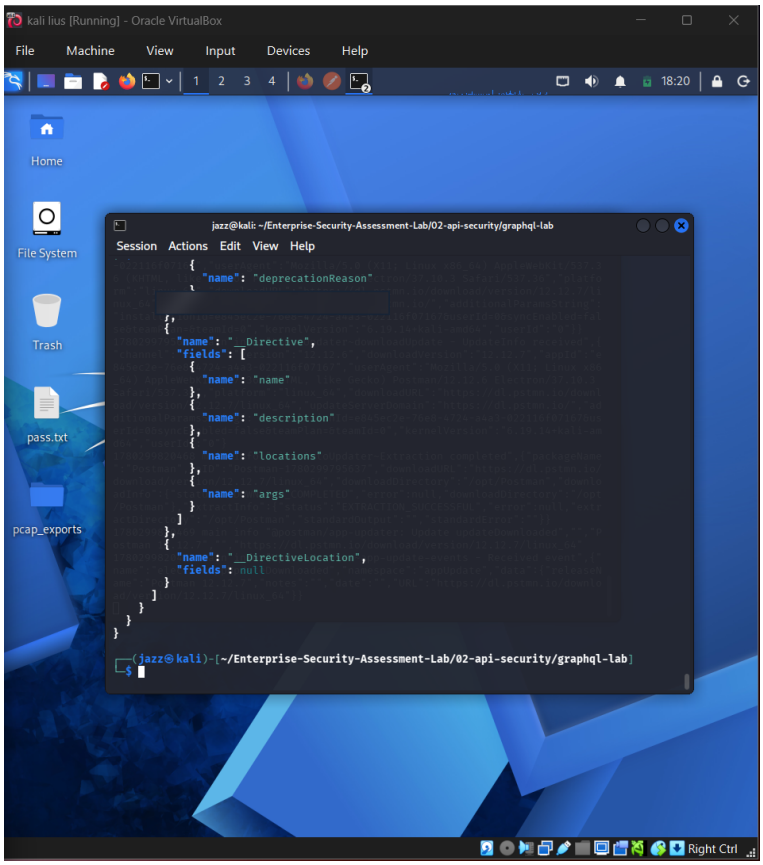

Figure 3 - Evidence of GraphQL schema discovery through introspection.

Evidence of GraphQL schema discovery through introspection.

Evidence 4: 13-graphql-user-data-query-test.png

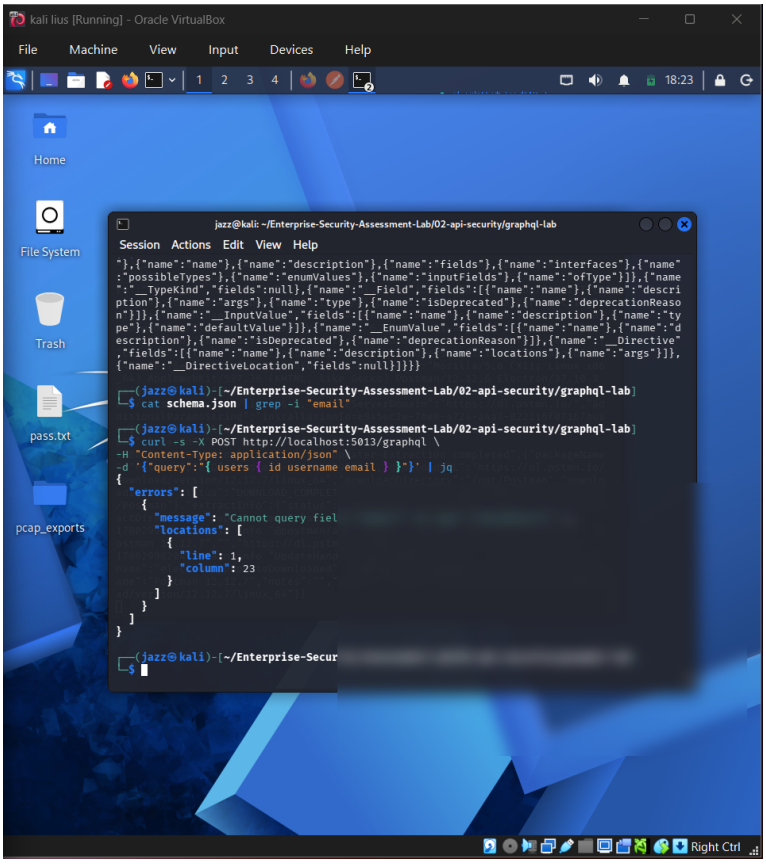

Figure 4 - Evidence of GraphQL query testing against user-data fields.

Evidence of GraphQL query testing against user-data fields.

3. Command and Workflow Used

```
# Endpoint availability check
GET /graphql

# Introspection-style query concept
{
  "__schema": {
    "types": {
      "name": "..."
    }
  }
}

# User data query concept
query {
  users {
    id
    email
    role
  }
}
```

Step-by-step workflow

- Verified that the DVGA GraphQL lab was running.
- Confirmed that the GraphQL endpoint responded to requests.

- Submitted introspection-style queries to review schema exposure.
- Queried discovered fields to assess returned user-data behaviour.
- Mapped results to GraphQL-specific security controls.

4. Professional Interpretation

GraphQL testing requires a different mindset from REST testing. A single endpoint can expose many types, queries, mutations, and nested relationships. Introspection can make mapping easier, but resolver-level authorization determines whether sensitive data is actually protected.

5. Security Risk and Impact

If GraphQL introspection, query depth, resolver authorization, and field-level restrictions are weak, attackers can map the schema, discover sensitive objects, request excessive nested data, or retrieve user fields they should not access.

6. Recommended Remediation and Hardening

- Disable introspection in production unless required and protected.
- Apply resolver-level authorization checks.
- Limit query depth, complexity, and cost.
- Return only necessary fields for each user role.
- Monitor unusual GraphQL query patterns.
- Use persisted queries where appropriate for production clients.

7. Framework Mapping

Framework Area	Mapping
OWASP API	API3, API4, API9 depending GraphQL behaviour
Security Control	Schema exposure, resolver authorization, query complexity limits
Evidence Result	GraphQL lab, endpoint, introspection, and query tests completed

8. Sanitized Output Style for GitHub

When documenting this evidence publicly, use placeholders instead of live values. The goal is to prove methodology and security understanding without exposing secrets or private data.

```
Authorization: Bearer <REDACTED_TOKEN>
Cookie: <REDACTED_COOKIE>
email: lab-user@example.com
password: REDACTED
object_id: <CONTROLLED_LAB_ID>
redirect_uri: <REDACTED_OR_LAB_CALLBACK>
```

9. GitHub Redaction Checklist

- Bearer tokens and JWT values must be blurred or replaced with <REDACTED_TOKEN>.
- Email addresses, phone numbers, user IDs, and object identifiers should be blurred when they are not required for understanding the evidence.
- Authorization headers, cookies, session values, OAuth codes, refresh tokens, access tokens, and client secrets must never be visible in public GitHub screenshots.
- If a screenshot contains personally identifiable data, crop the view or blur the affected rows before publication.

10. Publication Note

This report is designed for GitHub portfolio use. It describes authorized lab testing only and avoids production claims. The embedded screenshots have been treated as lab evidence and should remain redacted before upload.